Flask to FastAPI Migration Study

Receipt Verification Service — Implementation, Migration Challenges, and Performance Benchmarking

Prepared by: Vineet Singh Chilwal

Organisation: Accenture — Summer Internship

Date: 23 June 2026

Why Migrate from Flask to FastAPI?

Flask was the standard choice for Python APIs. Modern I/O-heavy workloads expose its synchronous limitations.

Flask — Legacy Approach

- Synchronous — one request blocks one thread entirely
- No built-in request/response validation
- No auto-generated API documentation
- Multiple OS worker processes needed to scale (Gunicorn)
- Background tasks require manual threads or external queues

FastAPI — Modern Approach

- Async/await — event loop handles many requests concurrently
- Pydantic models validate automatically, zero extra code
- Swagger UI generated at /docs with no configuration
- Single worker handles more concurrent traffic via async I/O
- BackgroundTasks built into the framework natively

Both were implemented to the same API contract so any performance difference is purely architectural.

Receipt Verification Service — What Was Built

The same REST API was implemented twice — once in Flask, once in FastAPI — connected to the same MongoDB Atlas cluster. Identical contract, different architecture.

Endpoint	Method	Input	Response
/health	GET	None	{"status": "ok"}
/upload-receipt	POST	Receipt image file (jpg/png/pdf, max 5MB)	{"receipt_id": "uuid"}
/extract	POST	{"receipt_id": "uuid"}	{"merchant": "...", "date": "...", "amount": "..."}
/validate	POST	{"receipt_id": "uuid"}	{"status": "Valid", "confidence": "92%"}

All POST endpoints require authentication — X-API-Key header or a JWT Bearer token obtained from POST /token.

Both services also expose GET /metrics (Prometheus monitoring) and POST /token (JWT issuance).

Phase 1 — Flask Implementation

Built first as the synchronous baseline. Every design decision here defined the contract the FastAPI version had to match exactly.

Architecture

- app.py — all routes, auth decorator, rate limiting, error handlers
- config/db.py — PyMongo + GridFS connection to MongoDB Atlas; pings Atlas on startup
- services/storage.py — GridFS file write/read + metadata collection
- services/receipt_service.py — extract and validate business logic
- model/predict.py — ML plug-in contract; stub returns hardcoded values now, real model replaces it later without touching any other file
- utils/file_validation.py — rejects files that are not jpg/jpeg/png/pdf or over 5MB

Test Coverage

- 32 unit tests covering all endpoints, both auth paths, rate limiting, metrics, background task, file validation
- MongoDB fully mocked at module level before any app code imports — runs completely offline
- python -m pytest tests/ -v → 32 passed

Local dev command: python app.py — Gunicorn (the production server in the Dockerfile) cannot run on Windows because it depends on fcntl, a Unix-only system module.

Phase 2 — FastAPI Implementation

Not a line-by-line port. FastAPI's patterns are genuinely different — async I/O, dependency injection, Pydantic validation, and a lifespan model for startup/shutdown.

Key Architectural Differences from Flask

- All routes: async def — every MongoDB call is awaited, not blocking
- Motor replaces PyMongo — async MongoDB driver; using sync PyMongo inside async routes would block the entire event loop
- Pydantic BaseModel on every request and response — validation is automatic, not manual
- require_api_key() is a dependency injected via Depends(), not a decorator
- config/db.py uses get_bucket() — a lazy singleton for GridFS; constructed on first use inside an async function, not at import time (Python 3.14 requirement)
- Lifespan context manager: Atlas ping + unique index creation on startup, connection close on shutdown

Test Coverage

- 35 unit tests — same surface as Flask plus async lifespan, BackgroundTasks, and OpenAPI schema
- Motor mocked using AsyncMock — plain MagicMock raises TypeError when awaited
- TestClient runs BackgroundTasks synchronously — no sleep() needed in tests
- python -m pytest tests/ -v → 35 passed

Local dev command: python -m uvicorn app:app --reload --port 8000 | Interactive docs: http://localhost:8000/docs

Migration Challenges — Technical Changes Required

Every layer of the application required deliberate changes — not just the route syntax.

Layer	Flask	FastAPI
Route functions	def, synchronous, returns dict	async def, must await every I/O call
Request validation	Manual checks, dataclasses, raise ValueError manually	Pydantic BaseModel — FastAPI validates automatically before route runs
Response validation	None — any dict can be returned	response_model= checks return shape before serialisation
Auth	@require_api_key custom decorator using @wraps	Depends(require_api_key) — async dependency resolved by FastAPI
Middleware	@app.before_request and @app.after_request hooks	Single async middleware function wrapping call_next()
Error handling	One @app.errorhandler per status code, {"error": ...} natively	Three @app.exception_handler functions needed to produce the same {"error": ...} shape FastAPI defaults to {"detail": ...}
Background tasks	threading.Thread(daemon=True) — fire-and-forget, no tracking	background_tasks.add_task() — built-in, runs after response sent

The auth and error handling conversions were the most time-consuming — FastAPI's defaults intentionally differ from Flask's.

Section 04

Production Readiness — Same Features on Both Services

Both services were hardened identically so that performance differences reflect the framework, not different feature coverage.

API Security

- X-API-Key header — static secret, never expires, service-to-service
- JWT Bearer token — POST /token exchanges API key for 60-min JWT (PyJWT, HS256)
- Either method accepted on all protected endpoints
- Rate limiting: 30 req/min per IP (Flask-Limiter / slowapi)
- /health and /metrics exempt from rate limiting

Observability

- Per-request ID (UUID) injected into every log line — traceable across concurrent requests
- GET /metrics exposes Prometheus-format counters: http_requests_total (method, endpoint, status) and http_request_duration_seconds (latency histogram)
- Recorded on every request including errors — no code in business logic

Background Tasks

- Both services run post-upload housekeeping after responding
- Flask: daemon thread — fire-and-forget, does not survive worker restart, needs Celery for reliable production use
- FastAPI: native BackgroundTasks — runs on event loop after response, no threading, no extra dependencies
- Difference in test behaviour: Flask test needs sleep(0.2) to wait for thread; FastAPI TestClient runs tasks synchronously before returning

Section 05

Performance Benchmark — Locust Load Test Results

Tool: Locust — realistic multi-step user simulation (upload → extract → validate → health, with think time)

Load: 20 concurrent simulated users, 5 users/second ramp-up, 30 seconds per service

Method: Sequential runs — Flask first, then FastAPI on the same machine to avoid CPU contention

Metric	Flask	FastAPI
Total requests completed	821	1500
Failures	0	0
Average latency	2193.1 ms	700.9 ms
P95 latency	2600.0 ms	2100.0 ms
P99 latency	3300.0 ms	3300.0 ms
Throughput	14.20 req/s	25.57 req/s

- FastAPI completed ~83% more requests in the same duration — throughput up ~80%

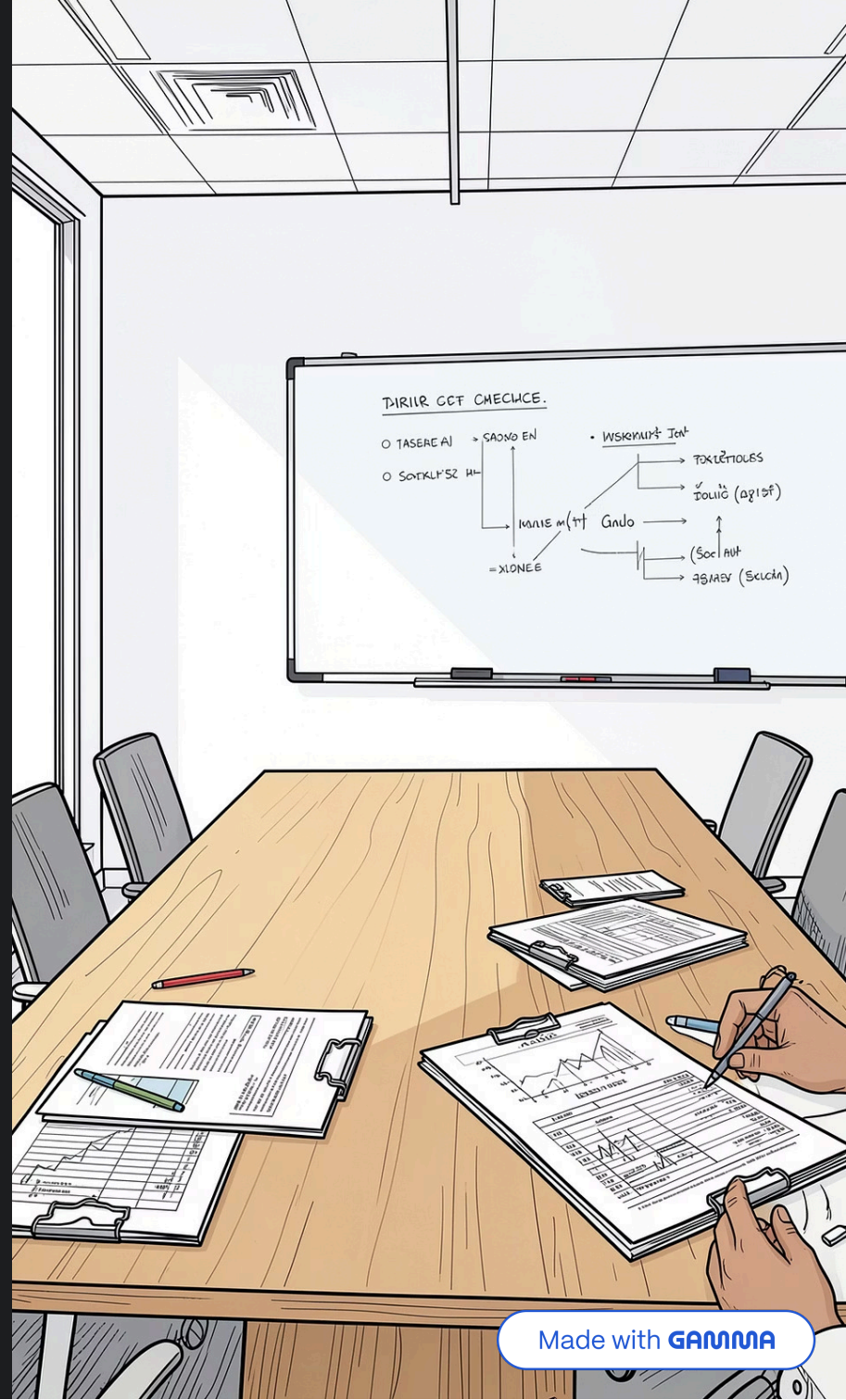
- Average latency reduced ~68% — async event loop continues serving requests while others wait on MongoDB

- P99 is identical on both (3300 ms) — tail latency is dominated by the MongoDB Atlas round-trip time, which no framework-level change can reduce; async helps concurrency, not individual worst-case requests

Conclusion

- **FastAPI performance advantage is real and measured — not theoretical.** 80% higher throughput and 68% lower average latency under identical concurrent load, with zero failures on both sides.
- **Migration cost is also real.** Four genuine technical blockers were encountered and resolved — driver version coupling, a Python 3.14 async event loop change, a Windows encoding mismatch, and a Swagger auth integration issue. None were FastAPI bugs; all were ecosystem and environment factors.
- **Tail latency (P99) equalised at 3300 ms on both frameworks.** Async improves how many requests move simultaneously — it cannot reduce the time a single slow database query takes.
- **Recommendation:** For I/O-heavy, concurrent Python APIs, FastAPI is the correct choice. The migration is worth the ecosystem learning curve — but the learning curve is real and should be budgeted for explicitly, not underestimated.

Flask app: 32/32 tests passing | FastAPI app: 35/35 tests passing
Benchmark: Locust, 20 concurrent users, results in benchmarks/results/locust/
Full documentation: Flask_to_FastAPI_Migration_Study_Documentation.docx



Thank You

Thank you for your time and attention.

Flask to FastAPI Migration Study
Receipt Verification Service — Implementation, Migration Challenges, and Performance Benchmarking